SIMATS ENGINEERING

ASSESSMENT TOOL 1: Data Interpretation

COURSE NAME: Query Processing for Data Science **COURSE CODE:** DSA0501

COURSE OUTCOME COVERED

CO1: Explain the fundamentals of data wrangling, its importance, processes, and tools. Understand different data formats such as CSV, JSON, and XML along with machine-readable data.

Assessment Tool Weightage: 40%

Total Marks: 20

Student Name: Answar Vijes D **Register No: 192324028**

Code of Conduct

I Answar Vijes D / 192324028 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Assessment Rubrics:

Data Understanding (4 Marks)	Data Analysis and Interpretation (4 Marks)	Use of Techniques (4 Marks)	Accuracy of Results (4 Marks)	Presentation of Findings (4 Marks)	Total (20 Marks)

Total Marks Awarded:

Signature of the Course Faculty

Answer Sheet - Assessment Tool 1: Data Interpretation

Course: Query Processing for Data Science (DSA0501) | Student: Answar Vijes D | Reg No: 192324028

Notes: All programs are written in Python 3 with Pandas. Since the question paper does not supply the input files, each answer first creates a small sample input file, then solves the task. The outputs shown were produced by
actually running the code. Currency amounts are written as Rs. inside code and outputs.

Q1. Employee Salary Analysis (employees.csv)

Task: Read the CSV file; Display employees earning more than Rs.50,000; Calculate average salary department-wise.

Approach: Load the file with pd.read_csv(), apply a Boolean mask on the Salary column, and use groupby() with mean().

Sample input (creates the file):

```
data = """EmpID,Name,Department,Salary
101,Arun,IT,65000
102,Bhavana,HR,48000
103,Charan,Finance,52000
104,Divya,IT,72000
105,Elango,HR,45000
106,Farah,Finance,58000
107,Gokul,Sales,39000
108,Hema,Sales,51000
"""
open("employees.csv", "w").write(data)
```

Program:

```
import pandas as pd

df = pd.read_csv("employees.csv")                    # 1. read the CSV print("All employees:\n",
df, "\n")

high = df[df["Salary"] > 50000]                      # 2. salary > 50,000 print("Employees earning
more than Rs.50,000:\n", high, "\n")

avg = df.groupby("Department")["Salary"].mean()      # 3. department-wise average print("Average salary by
department:\n", avg.round(2))
```

Output:

```
All employees:
   EmpID     Name Department  Salary
0    101     Arun         IT   65000
1    102  Bhavana         HR   48000
2    103   Charan    Finance   52000
3    104    Divya         IT   72000
4    105   Elango         HR   45000
5    106    Farah    Finance   58000
6    107    Gokul      Sales   39000
7    108     Hema      Sales   51000

Employees earning more than Rs.50,000:
   EmpID     Name Department  Salary
0    101     Arun         IT   65000
2    103   Charan    Finance   52000
3    104    Divya         IT   72000
5    106    Farah    Finance   58000
7    108     Hema      Sales   51000

Average salary by department:
 Department
Finance    55000.0
HR         46500.0
IT         68500.0
Sales      45000.0
Name: Salary, dtype: float64
```

Q2. Missing Data Handling

Task: Identify missing values; Replace missing marks with the column average; Display the cleaned dataset.
Approach: isnull().sum() counts the gaps; fillna(df.mean()) replaces each missing mark with the mean of its own column.

Sample input (creates the file):

```
data = """Name,Maths,Science,English
Anu,85,78,90
Bala,,82,75
Chitra,92,,88
Dinesh,70,65,
Esha,88,91,84
Farid,76,,79
"""
open("student_marks.csv", "w").write(data)
```

Program:

```
import pandas as pd

df = pd.read_csv("student_marks.csv")
print("Original data:\n", df, "\n")

print("Missing values per column:\n", df.isnull().sum(), "\n")

subjects = ["Maths", "Science", "English"]
df[subjects] = df[subjects].fillna(df[subjects].mean())       # column average df[subjects] = df[subjects].round(2)

print("Cleaned dataset:\n", df)
```

Output:

```
Original data:
      Name  Maths  Science  English
0     Anu   85.0     78.0     90.0
1    Bala    NaN     82.0     75.0
2  Chitra   92.0      NaN     88.0
3  Dinesh   70.0     65.0      NaN
4    Esha   88.0     91.0     84.0
5   Farid   76.0      NaN     79.0

Missing values per column:
 Name       0
Maths       1
Science     2
English     1
dtype: int64

Cleaned dataset:
      Name  Maths  Science  English
0     Anu   85.0     78.0     90.0
1    Bala   82.2     82.0     75.0
2  Chitra   92.0     79.0     88.0
3  Dinesh   70.0     65.0     83.2
4    Esha   88.0     91.0     84.0
5   Farid   76.0     79.0     79.0
```

Q3. Duplicate Record Removal

Task: Read the dataset; Find duplicate records; Remove duplicates; Display the updated dataset.
Approach: duplicated() flags repeated rows (first occurrence is kept); drop_duplicates() removes them.

Sample input (creates the file):

```
data = """CustomerID,Name,City,Phone
1,Ravi,Chennai,9876500001
2,Meena,Madurai,9876500002
3,Karthik,Coimbatore,9876500003
2,Meena,Madurai,9876500002
4,Latha,Salem,9876500004
1,Ravi,Chennai,9876500001
5,Nisha,Trichy,9876500005
"""
open("customers_dup.csv", "w").write(data)
```

Program:

```
import pandas as pd

df = pd.read_csv("customers_dup.csv")
print("Original dataset:\n", df, "\n")

dups = df[df.duplicated()]                                  # repeated rows print("Duplicate records
found:", len(dups))
print(dups, "\n")

df = df.drop_duplicates().reset_index(drop=True)    # keep first occurrence print("Updated dataset (duplicates
removed):\n", df)
```

Output:

```
Original dataset:
   CustomerID     Name          City        Phone
0           1     Ravi       Chennai  9876500001
1           2    Meena       Madurai  9876500002
2           3  Karthik    Coimbatore  9876500003
3           2    Meena       Madurai  9876500002
4           4    Latha        Salem  9876500004
5           1     Ravi       Chennai  9876500001
6           5    Nisha        Trichy  9876500005

Duplicate records found: 2
   CustomerID    Name      City        Phone
3           2   Meena   Madurai  9876500002
5           1    Ravi   Chennai  9876500001

Updated dataset (duplicates removed):
   CustomerID     Name          City        Phone
0           1     Ravi       Chennai  9876500001
1           2    Meena       Madurai  9876500002
2           3  Karthik    Coimbatore  9876500003
3           4    Latha        Salem  9876500004
4           5    Nisha        Trichy  9876500005
```

Q4. Sorting Dataset

Task: Sort products based on sales amount; Display top 10 highest-selling products.
Approach: sort_values(ascending=False) orders the rows by sales; head(10) picks the top ten.

Sample input (creates the file):

```
data = """Product,SalesAmount
Laptop,245000
Mobile,310000
Headphones,45000
Keyboard,18000
Monitor,98000
Printer,64000
Tablet,152000
Mouse,12500
Router,27000
Webcam,21000
Smartwatch,87000
Speaker,39000
"""
open("sales.csv", "w").write(data)
```

Program:

```
import pandas as pd

df = pd.read_csv("sales.csv")

sorted_df = df.sort_values(by="SalesAmount", ascending=False).reset_index(drop=True) print("Products sorted by sales
amount:\n", sorted_df, "\n")

top10 = sorted_df.head(10)
top10.index = top10.index + 1
print("Top 10 highest-selling products:\n", top10)
```

Output:

```
Products sorted by sales amount:
        Product  SalesAmount
0        Mobile       310000
1        Laptop       245000
2        Tablet       152000
3       Monitor        98000
4    Smartwatch        87000
5       Printer        64000
6    Headphones        45000
7       Speaker        39000
8        Router        27000
9        Webcam        21000
10     Keyboard        18000
11        Mouse        12500

Top 10 highest-selling products:
        Product  SalesAmount
1        Mobile       310000
2        Laptop       245000
3        Tablet       152000
4       Monitor        98000
5    Smartwatch        87000
6       Printer        64000
7    Headphones        45000
8       Speaker        39000
9        Router        27000
10       Webcam        21000
```

Q5. Filtering Records (Hospital Dataset)

Task: Display patients whose age is above 60; Display female patients; Count diabetic patients.
Approach: Use Boolean indexing for the age and gender filters and a conditional sum for the diabetic count.

Sample input (creates the file):

```
data = """PatientID,Name,Age,Gender,Diabetic
P01,Kamala,67,Female,Yes
P02,Rajesh,45,Male,No
P03,Sundar,72,Male,Yes
P04,Priya,34,Female,No
P05,Leela,61,Female,Yes
P06,Mohan,58,Male,No
P07,Revathi,29,Female,No
P08,Anand,65,Male,Yes
"""
open("patients.csv", "w").write(data)
```

Program:

```
import pandas as pd

df = pd.read_csv("patients.csv")

print("Patients above 60 years:\n", df[df["Age"] > 60], "\n")
print("Female patients:\n", df[df["Gender"] == "Female"], "\n") print("Number of diabetic patients:",
(df["Diabetic"] == "Yes").sum())
```

Output:

```
Patients above 60 years:
   PatientID     Name  Age  Gender Diabetic
0        P01   Kamala   67  Female      Yes
2        P03   Sundar   72    Male      Yes
4        P05    Leela   61  Female      Yes
7        P08    Anand   65    Male      Yes

Female patients:
   PatientID     Name  Age  Gender Diabetic
0        P01   Kamala   67  Female      Yes
3        P04    Priya   34  Female       No
4        P05    Leela   61  Female      Yes
6        P07  Revathi   29  Female       No

Number of diabetic patients: 4
```

Q6. Data Transformation (USD to INR)

Task: Convert prices into INR; Add a new column called "Price_INR".
Approach: Multiply the USD price column by an exchange rate (assumed Rs.83 per USD) and store it as a new column.

Sample input (creates the file):

```
data = """ProductID,Product,Price_USD
1,Laptop,950.00
2,Smartphone,620.50
3,Headphones,59.99
4,Keyboard,25.00
5,Monitor,210.75
"""
open("products_usd.csv", "w").write(data)
```

Program:

```
import pandas as pd

RATE = 83.0                                          # assumed: 1 USD = Rs.83

df = pd.read_csv("products_usd.csv")
df["Price_INR"] = (df["Price_USD"] * RATE).round(2)      # new column

print(df)
```

Output:

```
   ProductID        Product   Price_USD   Price_INR
0          1         Laptop      950.00    78850.00
1          2     Smartphone      620.50    51501.50
2          3     Headphones       59.99     4979.17
3          4       Keyboard       25.00     2075.00
4          5        Monitor      210.75    17492.25
```

Q7. Data Aggregation (Monthly Sales)

Task: Calculate total sales; Average sales; Maximum sales; Minimum sales.
Approach: Apply sum(), mean(), max() and min() on the Sales column; idxmax()/idxmin() give the corresponding month.

Sample input (creates the file):

```
data = """Month,Sales
January,120000
February,95000
March,134000
April,110500
May,148000
June,127500
"""
open("monthly_sales.csv", "w").write(data)
```

Program:

```
import pandas as pd

df = pd.read_csv("monthly_sales.csv")

print("Total sales   :", df["Sales"].sum())
print("Average sales:", round(df["Sales"].mean(), 2))
print("Maximum sales:", df["Sales"].max(), "(", df.loc[df["Sales"].idxmax(), "Month"], ")") print("Minimum sales:", df["Sales"].min(),
"(", df.loc[df["Sales"].idxmin(), "Month"], ")")
```

Output:

```
Total sales   : 735000
Average sales: 122500.0
Maximum sales: 148000 ( May )
Minimum sales: 95000 ( February )
```

Q8. Grouping Data

Task: Group employees by department and calculate: Total salary, Average salary, Employee count.
Approach: groupby('Department').agg() with named aggregations computes all three statistics in one step.

Sample input (creates the file):

```
data = """EmpID,Name,Department,Salary
101,Arun,IT,65000
102,Bhavana,HR,48000
103,Charan,Finance,52000
104,Divya,IT,72000
105,Elango,HR,45000
106,Farah,Finance,58000
107,Gokul,Sales,39000
108,Hema,Sales,51000
109,Imran,IT,60000
"""
open("employee_info.csv", "w").write(data)
```

Program:

```
import pandas as pd

df = pd.read_csv("employee_info.csv")

summary = df.groupby("Department").agg(
        Total_Salary=("Salary", "sum"),
        Average_Salary=("Salary", "mean"),
        Employee_Count=("EmpID", "count"),
).round(2)

print(summary)
```

Output:

```
            Total_Salary  Average_Salary  Employee_Count
Department
Finance           110000        55000.00               2
HR                 93000        46500.00               2
IT                197000        65666.67               3
Sales              90000        45000.00               2
```

Q9. Merge Two CSV Files

Task: Merge employees.csv and departments.csv using Department ID.
Approach: pd.merge() performs a SQL-style join on the common DeptID column.

Sample input (creates the file):

```
open("employees_m.csv", "w").write("""EmpID,Name,DeptID,Salary
1,Arun,10,65000
2,Bhavana,20,48000
3,Charan,30,52000
4,Divya,10,72000
5,Elango,20,45000
""")
open("departments.csv", "w").write("""DeptID,DeptName,Location
10,IT,Chennai
20,HR,Bengaluru
30,Finance,Mumbai
""")
```

Program:

```
import pandas as pd

emp = pd.read_csv("employees_m.csv")
dept = pd.read_csv("departments.csv")

merged = pd.merge(emp, dept, on="DeptID", how="inner")
print(merged)
merged.to_csv("merged_employees.csv", index=False)
```

Output:

```
   EmpID     Name  DeptID  Salary DeptName  Location
0      1     Arun      10   65000       IT   Chennai
1      2  Bhavana      20   48000       HR  Bengaluru
2      3   Charan      30   52000  Finance    Mumbai
3      4    Divya      10   72000       IT   Chennai
4      5   Elango      20   45000       HR  Bengaluru
```

Q10. Read JSON Data

Task: Read JSON data; Display student names; Find the topper.
Approach: json.load() reads the file into Python objects; max() with a key function finds the highest marks.

Sample input (creates the file):

```
import json
students = [
        {"id": 1, "name": "Aarthi",   "marks": 88},
        {"id": 2, "name": "Bharath", "marks": 76},
        {"id": 3, "name": "Chandru", "marks": 94},
        {"id": 4, "name": "Deepa",     "marks": 81},
]
json.dump(students, open("students_info.json", "w"), indent=2)
```

Program:

```
import json

with open("students_info.json") as f:
        students = json.load(f)

print("Student names:")
for s in students:
        print(" -", s["name"])

topper = max(students, key=lambda s: s["marks"])
print("\nTopper:", topper["name"], "with", topper["marks"], "marks")
```

Output:

```
Student names:
 - Aarthi
 - Bharath
 - Chandru
 - Deepa

Topper: Chandru with 94 marks
```

Q11. Convert CSV into JSON

Task: Convert the data in student.csv into JSON format.
Approach: Read the CSV with Pandas and export with to_json(orient='records'); each row becomes a JSON object.

Sample input (creates the file):

```
open("student.csv", "w").write("""RollNo,Name,Department,Marks
1,Aarthi,CSE,88
2,Bharath,ECE,76
3,Chandru,CSE,94
""")
```

Program:

```
import pandas as pd

df = pd.read_csv("student.csv")
df.to_json("student.json", orient="records", indent=4)

print(open("student.json").read())
```

Output:

```
[
    {
        "RollNo":1,
        "Name":"Aarthi",
        "Department":"CSE",
        "Marks":88
    },
    {
        "RollNo":2,
        "Name":"Bharath",
        "Department":"ECE",
        "Marks":76
    },
    {
        "RollNo":3,
        "Name":"Chandru",
        "Department":"CSE",
        "Marks":94
    }
]
```

Q12. JSON Data Filtering

Task: From employee.json display employees with Salary > Rs.60,000 and Department = IT.
Approach: Load the JSON into a DataFrame and combine two conditions with the & operator.

Sample input (creates the file):

```
import json
emps = [
  {"id":1,"name":"Arun","department":"IT","salary":65000},
{"id":2,"name":"Bhavana","department":"HR","salary":48000},
{"id":3,"name":"Divya","department":"IT","salary":72000},
{"id":4,"name":"Gokul","department":"Sales","salary":61000},
{"id":5,"name":"Imran","department":"IT","salary":55000}, ]
json.dump(emps, open("employee.json","w"), indent=2)
```

Program:

```
import pandas as pd

df = pd.read_json("employee.json")

result = df[(df["salary"] > 60000) & (df["department"] == "IT")] print("IT employees with
salary > Rs.60,000:\n", result)
```

Output:

```
IT employees with salary > Rs.60,000:
   id  name department  salary
0   1  Arun         IT   65000
2   3 Divya         IT   72000
```

Q13. Data Wrangling on an Online Retail Dataset

Task: Read the CSV file using Pandas; Identify and remove duplicate records; Handle missing values in the CustomerID column.
Approach: Read CustomerID as text, drop exact duplicates, then label missing IDs as 'Guest' so the sales rows are not lost (dropna is shown as the alternative).

Sample input (creates the file):

```
open("online_retail.csv", "w").write("""InvoiceNo,CustomerID,ProductName,Quantity,UnitPrice,Country,InvoiceDate INV001,17850,White Lantern,6,3.39,United
Kingdom,2025-01-05
INV001,17850,Metal Lantern,6,3.39,United Kingdom,2025-01-05
INV002,13047,Coffee Mug,12,1.25,United Kingdom,2025-01-06
INV002,13047,Coffee Mug,12,1.25,United Kingdom,2025-01-06
INV003,,Gift Bag,3,2.10,France,2025-01-07
INV004,12583,Notebook,10,0.85,France,2025-01-08
INV005,,Pen Set,5,4.50,Germany,2025-01-09
INV006,13748,Desk Lamp,2,15.95,United Kingdom,2025-01-09
INV006,13748,Desk Lamp,2,15.95,United Kingdom,2025-01-09
""")
```

Program:

```
import pandas as pd

df = pd.read_csv("online_retail.csv", dtype={"CustomerID": str})    # 1. read print("Shape before cleaning:",
df.shape)

print("Duplicate rows:", df.duplicated().sum())                    # 2. duplicates df =
df.drop_duplicates().reset_index(drop=True)
print("Shape after removing duplicates:", df.shape)

print("\nMissing CustomerID:", df["CustomerID"].isnull().sum())      # 3. missing IDs df["CustomerID"] =
df["CustomerID"].fillna("Guest")
# Alternative: df = df.dropna(subset=["CustomerID"])

print("\nCleaned dataset:\n", df)
```

Output:

```
Shape before cleaning: (9, 7)
Duplicate rows: 2
Shape after removing duplicates: (7, 7)

Missing CustomerID: 2

Cleaned dataset:
      InvoiceNo CustomerID      ProductName   Quantity   UnitPrice             Country InvoiceDate 0      INV001         17850
White Lantern              6          3.39   United Kingdom   2025-01-05 1      INV001         17850   Metal Lantern              6
3.39   United Kingdom   2025-01-05 2      INV002         13047       Coffee Mug            12          1.25   United Kingdom
2025-01-06 3      INV003         Guest          Gift Bag             3          2.10              France   2025-01-07 4      INV004
12583          Notebook            10          0.85              France   2025-01-08 5      INV005         Guest            Pen Set
5          4.50             Germany   2025-01-09 6      INV006         13748         Desk Lamp             2         15.95   United
Kingdom   2025-01-09
```

Q14. Read XML File

Task: Read XML data; Display employee names; Count total employees.
Approach: xml.etree.ElementTree parses the file; findall() returns every <employee> element.

Sample input (creates the file):

```
open("employee.xml", "w").write("""<?xml version="1.0"?>
<employees>
  <employee><id>1</id><name>Arun</name><salary>65000</salary></employee>
<employee><id>2</id><name>Bhavana</name><salary>48000</salary></employee>
<employee><id>3</id><name>Charan</name><salary>52000</salary></employee>
<employee><id>4</id><name>Divya</name><salary>72000</salary></employee>   </employees>
""")
```

Program:

```
import xml.etree.ElementTree as ET

root = ET.parse("employee.xml").getroot()
employees = root.findall("employee")

print("Employee names:")
for e in employees:
        print(" -", e.find("name").text)

print("\nTotal employees:", len(employees))
```

Output:

```
Employee names:
 - Arun
 - Bhavana
 - Charan
 - Divya

Total employees: 4
```

Q15. XML to DataFrame

Task: Convert products.xml into a Pandas DataFrame.
Approach: Loop through the <product> elements, build a list of dictionaries, and pass it to pd.DataFrame() (pd.read_xml('products.xml') is the one-line alternative).

Sample input (creates the file):

```
open("products.xml", "w").write("""<?xml version="1.0"?>
<products>
  <product><id>1</id><name>Laptop</name><price>55000</price></product>
<product><id>2</id><name>Mouse</name><price>450</price></product>
<product><id>3</id><name>Monitor</name><price>12500</price></product>  </products>
""")
```

Program:

```
import pandas as pd
import xml.etree.ElementTree as ET

root = ET.parse("products.xml").getroot()
rows = [{child.tag: child.text for child in p} for p in root.findall("product")]

df = pd.DataFrame(rows)
df["id"] = df["id"].astype(int)
df["price"] = df["price"].astype(float)

print(df)
print("\nData types:\n", df.dtypes)
# One-line alternative: df = pd.read_xml("products.xml")
```

Output:

```
   id    name    price
0   1  Laptop  55000.0
1   2   Mouse    450.0
2   3 Monitor  12500.0

Data types:
 id       int64
name      str
price   float64
dtype: object
```

Q16. XML Data Analysis (library.xml)

Task: Display: Total books; Books written after 2020; Books costing more than Rs.500.
Approach: Parse every <book>, convert year and price to numbers, then filter with list comprehensions.

Sample input (creates the file):

```
open("library.xml", "w").write("""<?xml version="1.0"?>
<library>
  <book><title>Python Basics</title><author>R. Kumar</author><year>2019</year><price>450</price></book>  <book><title>Data Wrangling</title><author>S. Iyer</author><year>2021</year><price>650</price></book>  <book><title>Machine Learning</title><author>A. Rao</author><year>2022</year><price>799</price></book>  <book><title>SQL Handbook</title><author>M. Nair</author><year>2020</year><price>350</price></book>  <book><title>Deep Learning</title><author>K. Das</author><year>2023</year><price>980</price></book> </library>
""")
```

Program:

```
import xml.etree.ElementTree as ET

books = []
for b in ET.parse("library.xml").getroot().findall("book"):          books.append({"title":
b.find("title").text,
                                        "year": int(b.find("year").text),
                                        "price": float(b.find("price").text)})

print("Total books:", len(books))

print("\nBooks written after 2020:")
for b in [b for b in books if b["year"] > 2020]:
        print(f" - {b['title']} ({b['year']})")

print("\nBooks costing more than Rs.500:")
for b in [b for b in books if b["price"] > 500]:
        print(f" - {b['title']} (Rs.{b['price']:.0f})")
```

Output:

```
Total books: 5

Books written after 2020:
 - Data Wrangling (2021)
 - Machine Learning (2022)
 - Deep Learning (2023)

Books costing more than Rs.500:
 - Data Wrangling (Rs.650)
 - Machine Learning (Rs.799)
 - Deep Learning (Rs.980)
```

Q17. Data Interpretation using Python (Marks List)

Task: Store marks in a list; Find: Highest mark, Lowest mark, Average mark, Number of students passed. **Approach:** Built-in max(), min(), sum()/len() and a comprehension; pass mark is taken as 40.

Program:

```
marks = [78, 45, 92, 38, 66, 81, 55, 29, 73, 88]
PASS_MARK = 40

print("Marks           :", marks)
print("Highest mark    :", max(marks))
print("Lowest mark     :", min(marks))
print("Average mark    :", sum(marks) / len(marks))
passed = len([m for m in marks if m >= PASS_MARK])
print("Students passed:", passed, "out of", len(marks))
```

Output:

```
Marks           : [78, 45, 92, 38, 66, 81, 55, 29, 73, 88]
Highest mark    : 92
Lowest mark     : 29
Average mark    : 64.5
Students passed: 8 out of 10
```

Q18. Dictionary-Based Analysis

Task: Store employee information using dictionaries; Display: Highest salary employee, Lowest salary employee, Average salary.

Approach: Keep employees in a list of dictionaries; use max()/min() with key=salary and sum()/len() for the average.

Program:

```
employees = [
    {"name": "Arun", "dept": "IT", "salary": 65000}, {"name": "Bhavana", "dept": "HR", "salary": 48000}, {"name": "Charan", "dept": "Finance", "salary": 52000}, {"name": "Divya", "dept": "IT", "salary": 72000}, {"name": "Gokul", "dept": "Sales", "salary": 39000}, ]

highest = max(employees, key=lambda e: e["salary"])
lowest = min(employees, key=lambda e: e["salary"])
average = sum(e["salary"] for e in employees) / len(employees)

print("Highest salary:", highest["name"], "-", highest["salary"]) print("Lowest salary :", lowest["name"], "-", lowest["salary"]) print("Average salary:", average)
```

Output:

```
Highest salary: Divya - 72000
Lowest salary : Gokul - 39000
Average salary: 55200.0
```

Q19. Database Schema Understanding

Task: Tables: Student(StudentID, Name, DeptID) and Department(DeptID, Department); Read both tables; Join them using DeptID; Display Student Name and Department.

Approach: Both tables are created in an in-memory SQLite database, read into DataFrames with read_sql_query(), and joined on DeptID.

Sample input (creates the file):

```
import sqlite3
con = sqlite3.connect("college.db")
con.executescript("""
DROP TABLE IF EXISTS Student; DROP TABLE IF EXISTS Department;
CREATE TABLE Department(DeptID INTEGER, Department TEXT);
CREATE TABLE Student(StudentID INTEGER, Name TEXT, DeptID INTEGER);
INSERT INTO Department VALUES (1,'CSE'),(2,'ECE'),(3,'Mechanical');
INSERT INTO Student VALUES (101,'Aarthi',1),(102,'Bharath',2),(103,'Chandru',1),
(104,'Deepa',3),(105,'Elango',2);
""")
con.commit(); con.close()
```

Program:

```
import sqlite3
import pandas as pd

con = sqlite3.connect("college.db")
student = pd.read_sql_query("SELECT * FROM Student", con) department =
pd.read_sql_query("SELECT * FROM Department", con) con.close()

print("Student table:\n", student, "\n")
print("Department table:\n", department, "\n")

joined = pd.merge(student, department, on="DeptID")
print("Student Name and Department:\n", joined[["Name", "Department"]])
```

Output:

```
Student table:
   StudentID     Name  DeptID
0        101   Aarthi       1
1        102  Bharath       2
2        103  Chandru       1
3        104    Deepa       3
4        105   Elango       2

Department table:
   DeptID   Department
0       1          CSE
1       2          ECE
2       3   Mechanical

Student Name and Department:
      Name   Department
0   Aarthi          CSE
1  Bharath          ECE
2  Chandru          CSE
3    Deepa   Mechanical
4   Elango          ECE
```

Q20. Customer Purchase Analysis (CSV)

Task: Read the CSV file; Find the customer who spent the highest amount; Display customers whose purchase amount is greater than the average; Save the filtered data into high_spenders.csv.
Approach: idxmax() finds the top spender, a Boolean mask compares each purchase with mean(), and to_csv() saves the result.

Sample input (creates the file):

```
open("customers.csv", "w").write("""CustomerID,Name,PurchaseAmount  C01,Aarthi,12500
C02,Bharath,4300
C03,Chandru,18900
C04,Deepa,7600
C05,Elango,22000
C06,Farah,3100
C07,Gokul,9800
""")
```

Program:

```
import pandas as pd

df = pd.read_csv("customers.csv")

top = df.loc[df["PurchaseAmount"].idxmax()]
print("Highest spender:", top["Name"], "-", top["PurchaseAmount"])

avg = df["PurchaseAmount"].mean()
print("Average purchase amount:", round(avg, 2), "\n")

high = df[df["PurchaseAmount"] > avg]
print("Customers above average:\n", high)

high.to_csv("high_spenders.csv", index=False)
print("\nSaved to high_spenders.csv")
```

Output:

```
Highest spender: Elango - 22000
Average purchase amount: 11171.43

Customers above average:
   CustomerID     Name  PurchaseAmount
0         C01   Aarthi           12500
2         C03  Chandru           18900
4         C05   Elango           22000

Saved to high_spenders.csv
```

Q21. Student Attendance Analysis

Task: Find students with attendance below 75%; Calculate the average attendance; Display the top 5 students with the highest attendance.
Approach: Filter with a Boolean mask, use mean() for the average and nlargest(5) for the top five.

Sample input (creates the file):

```
open("attendance.csv", "w").write("""StudentID,Name,Attendance S01,Aarthi,92
S02,Bharath,68
S03,Chandru,85
S04,Deepa,73
S05,Elango,96
S06,Farah,79
S07,Gokul,64
S08,Hema,88
""")
```

Program:

```
import pandas as pd

df = pd.read_csv("attendance.csv")

print("Students with attendance below 75%:\n", df[df["Attendance"] < 75], "\n") print("Average attendance: %.2f%%\n"
% df["Attendance"].mean())
print("Top 5 attendance:\n", df.nlargest(5, "Attendance").reset_index(drop=True))
```

Output:

```
Students with attendance below 75%:
   StudentID   Name  Attendance
1        S02  Bharath          68
3        S04    Deepa          73
6        S07    Gokul          64

Average attendance: 80.62%

Top 5 attendance:
   StudentID     Name  Attendance
0        S05   Elango          96
1        S01   Aarthi          92
2        S08    Hema          88
3        S03  Chandru          85
4        S06    Farah          79
```

Q22. Sales Trend Analysis

Task: Calculate daily sales; Find the day with maximum sales; Plot monthly sales using Matplotlib.
Approach: Group by Date to get daily totals, use idxmax() for the best day, and resample('M') to get monthly totals for the bar chart.

Sample input (creates the file):

```
import numpy as np, pandas as pd
rng = np.random.default_rng(7)
dates = pd.date_range("2025-01-01", "2025-04-30", freq="2D")
pd.DataFrame({
    "Date": dates,
    "Product": rng.choice(["Laptop","Mobile","Tablet"], len(dates)),    "Quantity": rng.integers(1,
6, len(dates)),
    "SalesAmount": rng.integers(5000, 60000, len(dates)),
}).to_csv("sales_trend.csv", index=False)
```

Program:

```
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

df = pd.read_csv("sales_trend.csv", parse_dates=["Date"])

daily = df.groupby("Date")["SalesAmount"].sum()
print("Daily sales (first 5 days):\n", daily.head(), "\n")

best = daily.idxmax()
print("Day with maximum sales:", best.date(), "->", daily.max())

monthly = df.set_index("Date")["SalesAmount"].resample("ME").sum() print("\nMonthly sales:\n",
monthly)

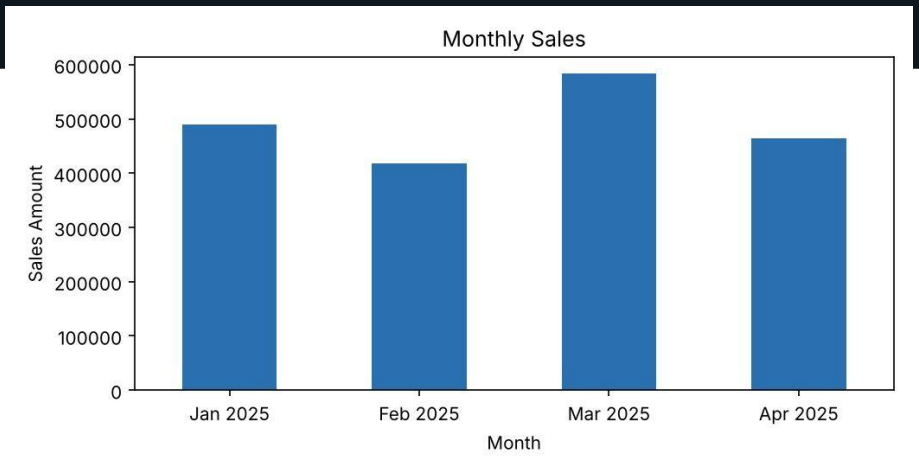
ax = monthly.plot(kind="bar", color="#2a6fb0", figsize=(7, 3.6))
ax.set_xticklabels([d.strftime("%b %Y") for d in monthly.index], rotation=0) ax.set_title("Monthly Sales");
ax.set_xlabel("Month"); ax.set_ylabel("Sales Amount") plt.tight_layout(); plt.savefig("monthly_sales.png", dpi=150)
```

Output:

```
Daily sales (first 5 days):
 Date
2025-01-01    29171
2025-01-03    38278
2025-01-05    29513
2025-01-07    40089
2025-01-09    35093
Name: SalesAmount, dtype: int64

Day with maximum sales: 2025-04-23 -> 59082

Monthly sales:
 Date
2025-01-31    489001
2025-02-28    417885
2025-03-31    584547
2025-04-30    465023
Freq: ME, Name: SalesAmount, dtype: int64
```



Q23. Data Standardization

Task: Standardize all department names (IT, I.T, Information Technology, info tech) to "IT"; Count employees in each department.
Approach: Normalise each label (lower-case, strip dots/spaces) and map every IT variant to 'IT'; then value_counts().

Sample input (creates the file):

```
open("emp_dept.csv", "w").write("""Name,Department
Arun,IT
Bhavana,HR
Charan,I.T
Divya,Information Technology
Elango,info tech
Farah,HR
Gokul,Finance
Hema,IT
""")
```

Program:

```
import pandas as pd

df = pd.read_csv("emp_dept.csv")
print("Before standardization:\n", df["Department"].value_counts(), "\n")

IT_VARIANTS = {"it", "informationtechnology", "infotech"}

def standardize(name):
    key = name.lower().replace(".", "").replace(" ", "")
    return "IT" if key in IT_VARIANTS else name.strip()

df["Department"] = df["Department"].apply(standardize)

print("After standardization:\n", df, "\n")
print("Employees in each department:\n", df["Department"].value_counts())
```

Output:

```
Before standardization:
 Department
IT                    2
HR                    2
I.T                   1
Information Technology  1
info tech             1
Finance               1
Name: count, dtype: int64

After standardization:
      Name  Department
0     Arun          IT
1  Bhavana          HR
2   Charan          IT
3    Divya          IT
4   Elango          IT
5    Farah          HR
6    Gokul     Finance
7     Hema          IT

Employees in each department:
 Department
IT          5
HR          2
Finance     1
Name: count, dtype: int64
```

Q24. Outlier Detection (Z score)

Task: Calculate mean and standard deviation; Identify outliers using the Z-score method; Remove outliers and display the cleaned dataset.
Approach: Z = (x - mean) / std. A mark with |Z| > 3 is treated as an outlier and filtered out.

Sample input (creates the file):

```
open("marks_outlier.csv", "w").write("Marks\n" + "\n".join(str(m) for m in
[72,68,75,70,66,74,71,69,73,67,72,70,68,74,71,69,73,70,72,5]) + "\n")
```

Program:

```
import pandas as pd

df = pd.read_csv("marks_outlier.csv")

mean, std = df["Marks"].mean(), df["Marks"].std()
print(f"Mean = {mean:.2f}, Standard deviation = {std:.2f}")

df["Z_score"] = (df["Marks"] - mean) / std
outliers = df[df["Z_score"].abs() > 3]
print("\nOutliers (|Z| > 3):\n", outliers.round(2))

clean = df[df["Z_score"].abs() <= 3].drop(columns="Z_score") print("\nCleaned dataset
(%d rows):" % len(clean))
print(clean["Marks"].tolist())
```

Output:

```
Mean = 67.45, Standard deviation = 14.90

Outliers (|Z| > 3):
   Marks  Z_score
19      5    -4.19

Cleaned dataset (19 rows):
[72, 68, 75, 70, 66, 74, 71, 69, 73, 67, 72, 70, 68, 74, 71, 69, 73, 70, 72]
```

Q25. String Cleaning

Task: Clean names; Remove extra spaces; Validate email addresses using regular expressions; Display invalid records.

Approach: str.strip(), a regex for extra spaces and str.title() clean the names; re.fullmatch() validates each email against a standard pattern.

Sample input (creates the file):

```
open("cust_db.csv", "w").write("""ID,Name,Email
1,  aRUN  kumar ,arun.kumar@gmail.com
2,BHAVANA  sri,bhavana@@yahoo.com
3,  charan  RAJ,charan.raj@outlook.com
4,divya  lakshmi ,divya.lakshmi@com
5,  ELANGO ,elango@mail.co.in
""")
```

Program:

```
import re
import pandas as pd

df = pd.read_csv("cust_db.csv")

df["Name"] = (df["Name"].str.strip()
                .str.replace(r"\s+", " ", regex=True)   # extra spaces
.str.title())                                  # proper case df["Email"] = df["Email"].str.strip().str.lower()

pattern = r"[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,}"
df["Valid_Email"] = df["Email"].apply(lambda e: bool(re.fullmatch(pattern, e)))

print("Cleaned data:\n", df, "\n")
print("Invalid email records:\n", df[~df["Valid_Email"]])
```

Output:

```
Cleaned data:
   ID          Name                 Email  Valid_Email
0   1  Arun Kumar  arun.kumar@gmail.com         True
1   2  Bhavana Sri  bhavana@@yahoo.com        False
2   3  Charan Raj  charan.raj@outlook.com
True
3   4  Divya Lakshmi  divya.lakshmi@com        False
4   5  Elango  elango@mail.co.in         True

Invalid email records:
   ID          Name                 Email  Valid_Email
1   2  Bhavana Sri  bhavana@@yahoo.com        False
3   4  Divya Lakshmi  divya.lakshmi@com        False
```

Q26. Data Type Conversion

Task: Convert Age and Salary (stored as strings) into numeric values; Handle invalid entries; Calculate the average salary.

Approach: pd.to_numeric(errors='coerce') turns invalid text into NaN; those rows are dropped before taking the mean.

Sample input (creates the file):

```
open("people.csv", "w").write("""Name,Age,Salary
Arun,28,"55,000"
Bhavana,thirty,48000
Charan,35,N/A
Divya,41,72000
Elango,29,"61,500"
""")
```

Program:

```
import pandas as pd

df = pd.read_csv("people.csv", dtype=str)
print("Before conversion:\n", df.dtypes, "\n")

df["Age"] = pd.to_numeric(df["Age"], errors="coerce")
df["Salary"] = pd.to_numeric(df["Salary"].str.replace(",", ""), errors="coerce")

print("Invalid entries became NaN:\n", df, "\n")
print("Data types after conversion:\n", df.dtypes, "\n")

valid = df.dropna(subset=["Salary"])
print("Average salary (valid entries only):", round(valid["Salary"].mean(), 2))
```

Output:

```
Before conversion:
 Name      str
Age        str
Salary     str
dtype: object

Invalid entries became NaN:
      Name   Age   Salary
0     Arun  28.0  55000.0
1  Bhavana   NaN  48000.0
2   Charan  35.0      NaN
3    Divya  41.0  72000.0
4   Elango  29.0  61500.0

Data types after conversion:
 Name           str
Age        float64
Salary     float64
dtype: object

Average salary (valid entries only): 59125.0
```

Q27. Date and Time Analysis

Task: Convert date strings into datetime objects; Find the number of orders placed each month; Display orders placed during weekends.

Approach: >= 5 pd.to_datetime() converts the strings; dt.to_period('M') groups by month; dt.dayofweek >= 5 selects Saturdays and Sundays.

Sample input (creates the file):

```
open("orders.csv", "w").write("""OrderID,OrderDate,Amount 1001,2025-01-04,2500
1002,2025-01-10,1800
1003,2025-01-18,3200
1004,2025-02-02,4100
1005,2025-02-14,900
1006,2025-02-22,2750
1007,2025-03-08,1500
1008,2025-03-12,3900
""")
```

Program:

```
import pandas as pd

df = pd.read_csv("orders.csv")
df["OrderDate"] = pd.to_datetime(df["OrderDate"])
print(df.dtypes, "\n")

print("Orders per month:\n", df.groupby(df["OrderDate"].dt.to_period("M")).size(), "\n")

df["Day"] = df["OrderDate"].dt.day_name()
print("Orders placed on weekends:\n", df[df["OrderDate"].dt.dayofweek >= 5])
```

Output:

```
OrderID                    int64
OrderDate         datetime64[us]
Amount                     int64
dtype: object

Orders per month:
  OrderDate
2025-01       3
2025-02       3
2025-03       2
Freq: M, dtype: int64

Orders placed on weekends:
        OrderID   OrderDate   Amount          Day
0        1001  2025-01-04       2500    Saturday
2        1003  2025-01-18       3200    Saturday
3        1004  2025-02-02       4100      Sunday
5        1006  2025-02-22       2750    Saturday
6        1007  2025-03-08       1500    Saturday
```

Q28. Missing Value Visualization (Healthcare)

Task: Count missing values in each column; Display the percentage of missing values; Replace missing values appropriately.

Approach: isnull().sum() and mean()*100 give counts and percentages (plotted as a bar chart); numeric gaps are filled with the median, categorical gaps with the mode.

Sample input (creates the file):

```
open("health.csv", "w").write("""PatientID,Age,Gender,BP,Cholesterol,Smoker
1,45,Male,130,210,Yes
2,,Female,120,,No
3,52,Female,,190,No
4,61,Male,145,230,
5,38,,118,175,No
6,,Male,125,,Yes
7,57,Female,138,205,No
""")
```

Program:

```
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

df = pd.read_csv("health.csv")

missing = df.isnull().sum()
percent = (df.isnull().mean() * 100).round(2)
print(pd.DataFrame({"Missing": missing, "Percent (%)": percent}), "\n")

ax = percent.plot(kind="bar", color="#c0504d", figsize=(6.5, 3.2))
ax.set_ylabel("% missing"); ax.set_title("Missing values per column")
plt.xticks(rotation=0); plt.tight_layout(); plt.savefig("missing_values.png", dpi=150)

for col in df.columns:
    if df[col].dtype.kind in "if":
        df[col] = df[col].fillna(df[col].median())    # numeric -> median    else:
        df[col] = df[col].fillna(df[col].mode()[0])   # text -> mode

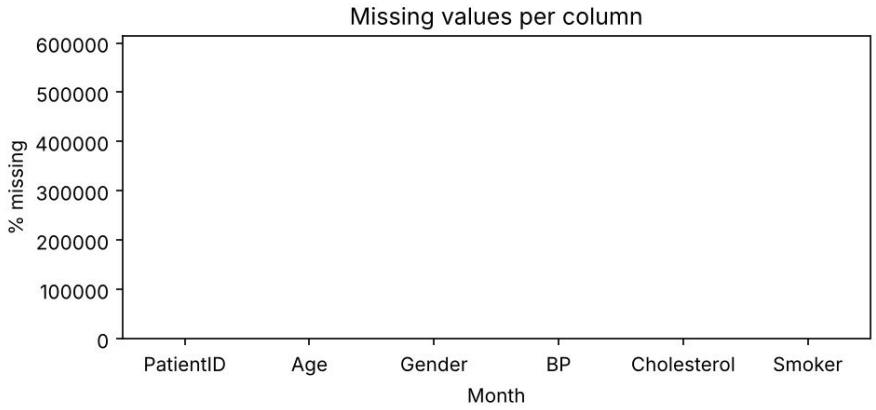
print("After replacing missing values:\n", df)
print("\nRemaining missing values:", int(df.isnull().sum().sum()))
```

Output:

```
                Missing      Percent     (%)
PatientID             0                0.00
Age                   2               28.57
Gender                1               14.29
BP                    1               14.29
Cholesterol           2               28.57
Smoker                1           14.29

After replacing missing values:
   PatientID   Age  Gender     BP  Cholesterol Smoker
0          1  45.0    Male  130.0        210.0    Yes
1          2  52.0  Female  120.0        205.0     No
2          3  52.0  Female  127.5        190.0     No
3          4  61.0    Male  145.0        230.0     No
4          5  38.0  Female  118.0        175.0     No
5          6  52.0    Male  125.0        205.0    Yes
6          7  57.0  Female  138.0        205.0     No

Remaining missing values: 0
```



Q29. Data Binning

Task: Categorize marks into Distinction (>=75), First Class (60-74), Second Class (50-59), Pass (40-49), Fail (<40); Display the number of students in each category.

Approach: pd.cut() with right=False places each mark into a half-open interval; value_counts() gives the category totals.

Sample input (creates the file):

```
open("marks_bin.csv", "w").write("""Name,Marks
Aarthi,88
Bharath,67
Chandru,54
Deepa,43
Elango,35
Farah,76
Gokul,59
Hema,62
Imran,81
Janani,39
""")
```

Program:

```
import pandas as pd

df = pd.read_csv("marks_bin.csv")

bins   = [0, 40, 50, 60, 75, 101]
labels = ["Fail", "Pass", "Second Class", "First Class", "Distinction"] df["Category"] = pd.cut(df["Marks"],
bins=bins, labels=labels, right=False)

print(df, "\n")
print("Students in each category:")
print(df["Category"].value_counts().reindex(labels[::-1]))
```

Output:

```
      Name  Marks        Category
0   Aarthi     88     Distinction
1  Bharath     67   First Class
2  Chandru     54  Second Class
3    Deepa     43            Pass
4   Elango     35            Fail
5    Farah     76     Distinction
6    Gokul     59  Second Class
7     Hema     62   First Class
8    Imran     81     Distinction
9   Janani     39            Fail

Students in each category:
Category
Distinction    3
First Class    2
Second Class   2
Pass           1
Fail           2
Name: count, dtype: int64
```

Q30. JSON Nested Objects

Task: Extract employee name, city, and state; Convert the data into a Pandas DataFrame.
Approach: pd.json_normalize() flattens nested dictionaries into columns such as address.city.

Sample input (creates the file):

```
import json
data = [
  {"id":1,"name":"Arun","address":{"street":"12 Anna Salai","city":"Chennai","state":"Tamil Nadu"}}, {"id":2,"name":"Bhavana","address":{"street":"8 MG Road","city":"Bengaluru","state":"Karnataka"}}, {"id":3,"name":"Charan","address":{"street":"45 Marine Drive","city":"Mumbai","state":"Maharashtra"}}, ]
json.dump(data, open("emp_nested.json","w"), indent=2)
```

Program:

```
import json
import pandas as pd

with open("emp_nested.json") as f:
        data = json.load(f)

flat = pd.json_normalize(data)
df = flat[["name", "address.city", "address.state"]]
df.columns = ["Name", "City", "State"]

print(df)
```

Output:

```
      Name       City          State
0     Arun    Chennai     Tamil Nadu
1  Bhavana  Bengaluru      Karnataka
2   Charan     Mumbai    Maharashtra
```

Q31. JSON API Simulation (Weather)

Task: Display: City name, Temperature, Humidity, Wind speed; Identify the hottest city.
Approach: Load the JSON list into a DataFrame, print the required columns and use idxmax() on Temperature.

Sample input (creates the file):

```
import json
w = [
  {"city":"Chennai","temperature":34.5,"humidity":72,"wind_speed":14.2},
{"city":"Delhi","temperature":38.1,"humidity":40,"wind_speed":9.8},
{"city":"Mumbai","temperature":32.0,"humidity":80,"wind_speed":18.5},
{"city":"Bengaluru","temperature":28.3,"humidity":60,"wind_speed":11.0}, ]
json.dump(w, open("weather.json","w"), indent=2)
```

Program:

```
import pandas as pd

df = pd.read_json("weather.json")
df.columns = ["City", "Temperature (C)", "Humidity (%)", "Wind Speed (km/h)"] print(df, "\n")

hot = df.loc[df["Temperature (C)"].idxmax()]
print("Hottest city:", hot["City"], "at", hot["Temperature (C)"], "C")
```

Output:

```
   City  Temperature (C)  Humidity (%)  Wind Speed (km/h)  0     Chennai
34.5                72                14.2  1     Delhi
38.1                40
9.8  2     Mumbai               32.0                80
18.5  3    Bengaluru
28.3                60                11.0

Hottest city: Delhi at 38.1 C
```

Q32. JSON Modification

Task: Add a new field called Grade based on marks; Save the updated JSON into a new file.
Approach: Loop over the records, compute Grade with an if/elif ladder and write the result with json.dump().

Sample input (creates the file):

```
import json
s = [
  {"id":1,"name":"Aarthi","marks":92},
  {"id":2,"name":"Bharath","marks":78},
  {"id":3,"name":"Chandru","marks":64},
  {"id":4,"name":"Deepa","marks":51},
  {"id":5,"name":"Elango","marks":33},
]
json.dump(s, open("students.json","w"), indent=2)
```

Program:

```
import json

def grade(m):
        if m >= 90: return "A+"
        if m >= 75: return "A"
        if m >= 60: return "B"
        if m >= 50: return "C"
        if m >= 40: return "D"
        return "F"

with open("students.json") as f:
        students = json.load(f)

for s in students:
        s["Grade"] = grade(s["marks"])

with open("students_graded.json", "w") as f:
        json.dump(students, f, indent=2)

print(open("students_graded.json").read())
```

Output:


```
[
  {
        "id": 1,
        "name": "Aarthi",
        "marks": 92,
        "Grade": "A+"
  },
  {
        "id": 2,
        "name": "Bharath",
        "marks": 78,
        "Grade": "A"
  },
  {
        "id": 3,
        "name": "Chandru",
        "marks": 64,
        "Grade": "B"
  },
  {
        "id": 4,
        "name": "Deepa",
        "marks": 51,
        "Grade": "C"
  },
  {
        "id": 5,
        "name": "Elango",
        "marks": 33,
        "Grade": "F"
  }
]
```


Q33. XML Student Records

Task: Extract all student names; Count students in each department; Display the topper from each department. **Approach:** Parse the XML into a DataFrame; groupby('dept') gives counts and idxmax() on marks gives each topper.

Sample input (creates the file):

```
open("students.xml", "w").write("""<?xml version="1.0"?>
<students>
  <student><name>Aarthi</name><dept>CSE</dept><marks>88</marks></student>
<student><name>Bharath</name><dept>ECE</dept><marks>76</marks></student>
<student><name>Chandru</name><dept>CSE</dept><marks>94</marks></student>
<student><name>Deepa</name><dept>ECE</dept><marks>81</marks></student>
<student><name>Elango</name><dept>MECH</dept><marks>69</marks></student>
<student><name>Farah</name><dept>CSE</dept><marks>72</marks></student>   </students>
""")
```

Program:

```
import pandas as pd
import xml.etree.ElementTree as ET

root = ET.parse("students.xml").getroot()
df = pd.DataFrame([{c.tag: c.text for c in s} for s in root.findall("student")]) df["marks"] = df["marks"].astype(int)

print("Student names:", df["name"].tolist(), "\n")
print("Students per department:\n", df["dept"].value_counts(), "\n")

toppers = df.loc[df.groupby("dept")["marks"].idxmax()]
print("Topper of each department:\n", toppers.reset_index(drop=True))
```

Output:

```
Student names: ['Aarthi', 'Bharath', 'Chandru', 'Deepa', 'Elango', 'Farah']

Students per department:
 dept
CSE     3
ECE     2
MECH    1
Name: count, dtype: int64

Topper of each department:
      name  dept  marks
0  Chandru   CSE     94
1    Deepa   ECE     81
2   Elango  MECH     69
```

Q34. XML Data Update

Task: Increase every product price by 10%; Save the updated XML file.
Approach: Parse the tree, multiply each <price> text by 1.10, and write it back with tree.write().

Sample input (creates the file):

```
open("prices.xml", "w").write("""<?xml version="1.0"?>
<products>
  <product><name>Laptop</name><price>55000</price></product>
<product><name>Mouse</name><price>450</price></product>
<product><name>Monitor</name><price>12500</price></product>  </products>
""")
```

Program:

```
import xml.etree.ElementTree as ET

tree = ET.parse("prices.xml")

for p in tree.getroot().findall("product"):
    old = float(p.find("price").text)
    new = round(old * 1.10, 2)
    p.find("price").text = str(new)
    print(f"{p.find('name').text}: {old} -> {new}")

ET.indent(tree)                                    # pretty print
tree.write("prices_updated.xml", encoding="utf-8", xml_declaration=True)

print("\nSaved file prices_updated.xml:\n")
print(open("prices_updated.xml").read())
```

Output:

```
Laptop: 55000.0 -> 60500.0
Mouse: 450.0 -> 495.0
Monitor: 12500.0 -> 13750.0

Saved file prices_updated.xml:

<?xml version='1.0' encoding='utf-8'?>
<products>
  <product>
        <name>Laptop</name>
        <price>60500.0</price>
  </product>
  <product>
        <name>Mouse</name>
        <price>495.0</price>
  </product>
  <product>
        <name>Monitor</name>
        <price>13750.0</price>
  </product>
</products>
```

Q35. Relational Database Schema Analysis

Task: Tables: Customers(CustomerID, Name) and Orders(OrderID, CustomerID, Amount); Join both tables; Find customers who placed more than three orders; Calculate total purchase amount for each customer.
Approach: merge() joins on CustomerID; groupby() gives the order count and total amount; customers with count > 3 are filtered.

Sample input (creates the file):

```
import pandas as pd
pd.DataFrame({"CustomerID":[1,2,3,4],"Name":["Arun","Bhavana","Charan","Divya"]}).to_csv("customers_tbl.csv", index=False) pd.DataFrame({"OrderID":range(1,13),
 "CustomerID":[1,1,1,1,2,2,3,3,3,3,3,4],
 "Amount":[500,750,300,1200,400,650,900,150,350,800,1000,2200]}).to_csv("orders_tbl.csv", index=False)
```

Program:

```
import pandas as pd

customers = pd.read_csv("customers_tbl.csv")
orders = pd.read_csv("orders_tbl.csv")

joined = pd.merge(customers, orders, on="CustomerID")
print("Joined table (first 5 rows):\n", joined.head(), "\n")

summary = joined.groupby(["CustomerID", "Name"]).agg(
        Orders=("OrderID", "count"), Total_Amount=("Amount", "sum")).reset_index()

print("Customers who placed more than 3 orders:\n", summary[summary["Orders"] > 3], "\n") print("Total purchase amount per
customer:\n", summary[["Name", "Total_Amount"]])
```

Output:

```
Joined table (first 5 rows):
   CustomerID    Name  OrderID  Amount
0           1    Arun        1     500
1           1    Arun        2     750
2           1    Arun        3     300
3           1    Arun        4    1200
4           2  Bhavana        5     400

Customers who placed more than 3 orders:
   CustomerID   Name  Orders  Total_Amount
0           1   Arun       4          2750
2           3  Charan       5          3200

Total purchase amount per customer:
      Name  Total_Amount
0     Arun          2750
1  Bhavana          1050
2   Charan          3200
3    Divya          2200
```

Q36. Multi-Table Join

Task: Tables: Employee, Department, Salary; Merge all tables; Find employees earning above department average; Display department-wise highest salary.
Approach: Chain two merge() calls, use groupby().transform('mean') to attach each department's average, then compare.

Sample input (creates the file):

```
import pandas as pd
pd.DataFrame({"EmpID":[1,2,3,4,5,6],"Name":["Arun","Bhavana","Charan","Divya","Elango","Farah"],
                "DeptID":[10,20,10,30,20,10]}).to_csv("employee_tbl.csv", index=False)
pd.DataFrame({"DeptID":[10,20,30],"DeptName":["IT","HR","Finance"]}).to_csv("department_tbl.csv", index=False) pd.DataFrame({"EmpID":[1,2,3,4,5,6],"Salary":
[65000,48000,72000,52000,45000,58000]}).to_csv("salary_tbl.csv", index=False)
```

Program:

```
import pandas as pd

emp = pd.read_csv("employee_tbl.csv")
dept = pd.read_csv("department_tbl.csv")
sal = pd.read_csv("salary_tbl.csv")

full = emp.merge(dept, on="DeptID").merge(sal, on="EmpID") print("Merged data:\n",
full, "\n")

full["Dept_Avg"] = full.groupby("DeptName")["Salary"].transform("mean").round(2)
print("Employees earning above department average:\n",
      full[full["Salary"] > full["Dept_Avg"]][["Name", "DeptName", "Salary", "Dept_Avg"]], "\n")

top = full.loc[full.groupby("DeptName")["Salary"].idxmax(), ["DeptName", "Name", "Salary"]] print("Department-wise highest salary:\n",
top.reset_index(drop=True))
```

Output:

```
Merged data:
   EmpID    Name  DeptID DeptName  Salary
0      1    Arun      10       IT   65000
1      2 Bhavana      20       HR   48000
2      3  Charan      10       IT   72000
3      4   Divya      30  Finance   52000
4      5  Elango      20       HR   45000
5      6   Farah      10       IT   58000

Employees earning above department average:
      Name DeptName  Salary  Dept_Avg
1 Bhavana       HR   48000   46500.0
2  Charan       IT   72000   65000.0

Department-wise highest salary:
  DeptName    Name  Salary
0  Finance   Divya   52000
1       HR Bhavana   48000
2       IT  Charan   72000
```

Q37. Data Validation

Task: Detect invalid records (negative salaries, invalid ages, empty names); Remove incorrect entries; Generate a validation report.

Approach: Three Boolean rules (salary < 0, age outside 18-65, blank name) flag bad rows; the clean rows are kept and a summary report is printed.

Sample input (creates the file):

```
open("emp_validation.csv", "w").write("""EmpID,Name,Age,Salary 1,Arun,28,55000
2,Bhavana,34,-48000
3,,41,52000
4,Divya,150,72000
5,Elango,29,45000
6,Farah,-5,58000
7,Gokul,38,61000
""")
```

Program:

```
import pandas as pd

df = pd.read_csv("emp_validation.csv")

bad_salary = df["Salary"] < 0
bad_age = (df["Age"] < 18) | (df["Age"] > 65)
bad_name = df["Name"].isnull() | (df["Name"].astype(str).str.strip() == "") invalid = bad_salary | bad_age |
bad_name

print("Invalid records:\n", df[invalid], "\n")
clean = df[~invalid].reset_index(drop=True)
print("Cleaned data:\n", clean, "\n")

print("===== VALIDATION REPORT =====")
print("Total records :", len(df))
print("Negative salaries :", bad_salary.sum())
print("Invalid ages :", bad_age.sum())
print("Empty names :", bad_name.sum())
print("Invalid records :", invalid.sum())
print("Valid records :", len(clean))
```

Output:

```
Invalid records:
   EmpID   Name  Age  Salary
1      2 Bhavana   34  -48000
2      3    NaN   41   52000
3      4  Divya  150   72000
5      6  Farah   -5   58000

Cleaned data:
   EmpID   Name  Age  Salary
0      1   Arun   28   55000
1      5 Elango   29   45000
2      7  Gokul   38   61000

===== VALIDATION REPORT =====
Total records : 7
Negative salaries : 1
Invalid ages : 2
Empty names : 1
Invalid records : 4
Valid records : 3
```

Q38. Exploratory Data Analysis (EDA)

Task: From sales.csv calculate: Mean, Median, Mode, Standard deviation, Variance; Correlation between Quantity and Sales.

Approach: Use the descriptive-statistics methods of Pandas and corr() for the Pearson correlation coefficient.

Sample input (creates the file):

```
open("sales_eda.csv", "w").write("""Quantity,Sales
5,2500
8,4100
3,1450
8,4000
12,6200
7,3600
10,5100
8,4150
""")
```

Program:

```
import pandas as pd

df = pd.read_csv("sales_eda.csv")

for col in ["Quantity", "Sales"]:
    print(f"--- {col} ---")
    print("Mean :", round(df[col].mean(), 2)) print("Median :",
    df[col].median())
    print("Mode :", df[col].mode().tolist()) print("Standard deviation:",
    round(df[col].std(), 2)) print("Variance :", round(df[col].var(), 2), "\n")

print("Correlation (Quantity vs Sales):", round(df["Quantity"].corr(df["Sales"]), 4))
```

Output:

```
--- Quantity ---
Mean                : 7.62
Median              : 8.0
Mode                : [8]
Standard deviation: 2.77
Variance            : 7.7

--- Sales ---
Mean                : 3887.5
Median              : 4050.0
Mode                : [1450, 2500, 3600, 4000, 4100, 4150, 5100, 6200] Standard deviation:
1458.66
Variance            : 2127678.57

Correlation (Quantity vs Sales): 0.9995
```

Q39. Comprehensive Data Wrangling Project (E-Commerce Orders)

Task: Read the dataset; Remove duplicate rows; Handle missing values; Convert dates into datetime format; Calculate Total Price = Quantity x Price; Identify the top-selling product.

Approach: A full wrangling pipeline: read, de-duplicate, impute (median / mode), parse dates, derive Total_Price and aggregate by product.

Sample input (creates the file):

```
open("ecommerce_orders.csv", "w").write("""OrderID,Customer,Product,Category,Quantity,Price,OrderDate,City,PaymentMode
1,Arun,Laptop,Electronics,1,55000,2025-01-05,Chennai,Card
2,Bhavana,Mouse,Accessories,3,450,2025-01-06,Madurai,UPI
3,Charan,Keyboard,Accessories,2,900,2025-01-07,Salem,UPI
3,Charan,Keyboard,Accessories,2,900,2025-01-07,Salem,UPI
4,Divya,Mouse,Accessories,,450,2025-01-09,Chennai,Card
5,Elango,Monitor,Electronics,1,12500,2025-01-10,,Cash
6,Farah,Mouse,Accessories,4,450,2025-01-12,Trichy,
7,Gokul,Laptop,Electronics,1,55000,2025-01-15,Coimbatore,Card
""")
```

Program:

```
import pandas as pd

df = pd.read_csv("ecommerce_orders.csv")                    # 1. read print("Rows read:", len(df))

df = df.drop_duplicates().reset_index(drop=True)            # 2. duplicates print("Rows after removing
duplicates:", len(df))

print("\nMissing values:\n", df.isnull().sum()[lambda s: s > 0])
df["Quantity"] = df["Quantity"].fillna(round(df["Quantity"].median())).astype(int)   # 3. missing df["City"] =
df["City"].fillna("Unknown")
df["PaymentMode"] = df["PaymentMode"].fillna(df["PaymentMode"].mode()[0])

df["OrderDate"] = pd.to_datetime(df["OrderDate"])           # 4. dates df["Total_Price"] = df["Quantity"] *
df["Price"]                 # 5. total price

print("\nCleaned dataset:\n", df.drop(columns=["Customer", "Category"]), "\n")

by_qty = df.groupby("Product")["Quantity"].sum().sort_values(ascending=False) print("Units sold per product:\n",
by_qty, "\n")           # 6. top seller print("Top-selling product (by quantity):", by_qty.idxmax())
by_rev = df.groupby("Product")["Total_Price"].sum()
print("Top-selling product (by revenue) :", by_rev.idxmax())
```

Output:

```
Rows read: 8
Rows after removing duplicates: 7

Missing values:
 Quantity        1
City             1
PaymentMode      1
dtype: int64

Cleaned dataset:
        OrderID     Product   Quantity   Price   OrderDate            City  PaymentMode   Total_Price  0           1      Laptop
1   55000  2025-01-05        Chennai            Card            55000  1           2       Mouse             3      450  2025-01-06
Madurai              UPI              1350  2           3    Keyboard             2      900  2025-01-07          Salem              UPI
1800  3            4        Mouse              2      450  2025-01-09        Chennai             Card              900  4            5
Monitor              1   12500  2025-01-10        Unknown             Cash             12500  5           6       Mouse             4
450  2025-01-12         Trichy             Card             1800  6           7       Laptop              1   55000  2025-01-15   Coimbatore
Card             55000

Units sold per product:
 Product
Mouse          9
Keyboard       2
Laptop         2
Monitor        1
Name: Quantity, dtype: int64

Top-selling product (by quantity): Mouse
Top-selling product (by revenue) : Laptop
```

Q40. Read Multiple CSV Files and Merge into One Master Dataset

Task: Read multiple CSV files from a folder and merge them into one master dataset.
Approach: glob finds every *.csv in the folder; each file is read and pd.concat(ignore_index=True) stacks them into one DataFrame.

Sample input (creates the file):

```
import os
os.makedirs("data", exist_ok=True)
open("data/sales_jan.csv","w").write("Product,Qty,Amount\nPen,10,100\nBook,5,500\n")
open("data/sales_feb.csv","w").write("Product,Qty,Amount\nPen,20,200\nBag,2,900\n")
open("data/sales_mar.csv","w").write("Product,Qty,Amount\nBook,7,700\nBottle,4,400\n")
```

Program:

```
import glob, os
import pandas as pd

files = sorted(glob.glob("data/*.csv"))
print("Files found:", [os.path.basename(f) for f in files], "\n")

frames = []
for f in files:
        part = pd.read_csv(f)
        part["SourceFile"] = os.path.basename(f)        # remember where each row came from        frames.append(part)

master = pd.concat(frames, ignore_index=True)
master.to_csv("master_dataset.csv", index=False)

print("Master dataset:\n", master)
print("\nShape:", master.shape)
```

Output:

```
Files found: ['sales_feb.csv', 'sales_jan.csv', 'sales_mar.csv']

Master dataset:
     Product  Qty  Amount       SourceFile
0       Pen   20     200   sales_feb.csv
1       Bag    2     900   sales_feb.csv
2       Pen   10     100   sales_jan.csv
3      Book    5     500   sales_jan.csv
4      Book    7     700   sales_mar.csv
5    Bottle    4     400   sales_mar.csv

Shape: (6, 4)
```